

Milestone 3 DONE

EML Viewer Chrome Extension · Inline Images + Attachment List

In plain English

Emails now look like emails. The two visible additions:

- **Embedded images show up.** Company logos and signature graphics that used to appear as broken-image icons now render properly.
- **Attachments are listed.** Real attachments (a PDF, a spreadsheet, a `note.txt`) appear as a tidy list at the top of the email, with their type and size.

And it all still happens with **zero internet access** — the images are pulled out of the email file itself, not downloaded from anywhere.

How embedded images work (the "cid:" trick)

When a company puts a logo in an email, the image isn't typed into the HTML — it's shipped as a *separate part* of the email file and the HTML just points to it with a label, like ``. "CID" means **Content-ID** — an internal reference, not a web address.

A normal browser has no idea what `cid:logo@acme` means, so it shows a broken image. Our fix: during import we find each embedded part, convert its raw bytes into a **self-contained data: URL** (the image encoded directly as text), and swap every `cid:` reference for that. The result is a body that carries its own images inside it — nothing to fetch, so nothing can leak out.

Order matters: resolve, then sanitize

We resolve the `cid:` images *first*, then run the safety sanitizer. That ordering is deliberate: the sanitizer is configured to keep `data:` images, so the resolved logos survive the cleaning pass. We handle images referenced three ways — in `<img>` tags, in inline `style="background:url(...)"`, and inside `<style>` blocks.

Inline images vs. attachments

One subtlety: a logo is technically an "attachment" too, but you don't want your signature graphic cluttering the attachment list. So we tag each part: if it's referenced inline in the body (has a Content-ID or is marked "inline"), it shows **in the body only**. Everything else (the real PDF, the spreadsheet)

shows **in the attachment list**. The test email demonstrates both at once — an inline logo and a separate `note.txt`.

What we built this milestone

File	Role
<code>cid-resolver.js</code>	Finds inline parts, encodes them as <code>data:</code> URLs, rewrites <code>cid:</code> references in tags, style attributes, and style blocks.
<code>import.js</code>	Pipeline now: parse → resolve cid → sanitize → save. Also tags which parts are inline.
<code>viewer page</code>	New attachment list (filename · type · size) above the body, showing only non-inline attachments.

Proof it works (automated)

- **20 unit tests pass** (up from 11). New tests cover Content-ID normalization (brackets, case), `<img>` and CSS `url(cid:)` rewriting, and leaving unknown references alone.
- **Real end-to-end test:** the actual `inline-image.eml` and `malicious.eml` sample files are run through the genuine parse → resolve → sanitize pipeline, confirming the logo becomes a `data: image`, the `note.txt` stays a separate attachment, and the malicious one is still neutralized.

✓ Your check in Chrome (~2 minutes)

1. `chrome://extensions` → **reload** the EML Viewer.
2. Click the icon → drop `samples/inline-image.eml`. **Expect:**
 - A **blue square logo renders** in the body (no broken-image icon).
 - An **"Attachments (1)"** list appears above the body showing `note.txt` with its type and size.
3. (Re-check) Drop `samples/basic.eml` — still reads cleanly, no attachment list (it has none).

If the logo shows and `note.txt` is listed, Milestone 3 is confirmed. Final stretch: **Milestone 4** — clicking an attachment to preview (images/PDF) or download, plus error states and polish.

Known gaps until next milestone

- Attachments are **listed but not yet clickable** — opening/previewing/downloading them is Milestone 4.

- No PDF or image preview tabs yet (Milestone 4).
- Very large inline images make the stored body bigger (data: URLs add ~33%); fine within the 50 MB file cap, noted for later optimization.